Succinct trees

Advanced Data Structures – Summer 2026 – Lecture 2

Ragnar {Groot Koerkamp}, Stefan Walzer, Stefan Hermann

2026-05-04 Mon 14:00

Previous slides: Rank & Select, Elias-Fano

- Rank: counting 1 bits
  - 2 levels of blocks of length $\log^2 n$ and $\log n$
- Select: finding 1 bits
  - 2 levels of blocks with $\log^2 n$ 1-bits and $\sqrt{\log n}$ 1-bits.
- There exist succinct $o(n)$ space data structures for rank and select.
- Elias-Fano: efficient encoding of sorted list of integers
  - Split in high and ℓ-bit low part; ≈ 0.5 bit/elem overhead.
  - Encode high parts using *stars & bars*:
    1 for each element, 0 each time a multiple of 2^ℓ is crossed.

Today: Succinct representation of trees

A.k.a.: applications of rank and select.

- What is a tree?
- How many trees on n nodes are there?
- How do we encode the tree?
  - 3 options: LOUDS (BFS),
  - balanced parentheses (in-order DFS),
  - DFUDS (pre-order DFS)
- Operations on trees
  - degree, parent, i'th child, subtree size

Application: in-place json parsing

Ordered trees

Definition 1.

An *ordered tree* is a tree with:

- n nodes, $n - 1$ edges,
- a single root,
- an order on the children of each node.

Question 1.

How many ordered trees are there at least? At most?

Bijection to balanced parentheses

Theorem 1 Balanced parentheses.

There is a bijection between ordered trees on n nodes and *balanced parentheses (BP) sequences* of $n-1$ "(" and ")", where each "(" has a matching ")".

Proof

Bijection to balanced parentheses

Theorem 1 Balanced parentheses.

There is a bijection between ordered trees on n nodes and *balanced parentheses (BP) sequences* of $n-1$ "(" and ")", where each "(" has a matching ")".

Proof

"Trace" the outline of the tree in DFS-order: each time an edge goes down, write "(", and each time an edge goes up, write ")".

Upper bound

Theorem 2 Upper bound.

There are $o(4^n)$ ordered trees on n nodes.

Proof

Upper bound

Theorem 2 Upper bound.

There are $o(4^n)$ ordered trees on n nodes.

Proof

The number of BP sequences is at most

$$\#\text{trees} \leq \binom{2(n-1)}{n-1} = \Theta\left(\frac{2^{2n}}{\sqrt{n}}\right).$$

Or simpler: each of the $2(n-1)$ positions is (or), giving $\leq 2^{2(n-1)} < 4^n$.

Lower bound

Theorem 3 Lower bound.

There are $\Omega(4^n/n^{1.5})$ ordered trees on n nodes.

Proof

Lower bound

Theorem 3 Lower bound.

There are $\Omega(4^n/n^{1.5})$ ordered trees on n nodes.

Proof

Partition the $\binom{2(n-1)}{n-1}$ possible BP sequences in equivalence classes under rotation of the string. Take a random element of any class. Find the prefix with the largest value of $\texttt{count}_{)}(i) - \texttt{count}_{(}(i)$, and consider the rotation starting after this prefix. This is a BP. Thus, each class has size $\leq 2(n-1)$, and contains at least one BP:

Lower bound

Theorem 3 Lower bound.

There are $\Omega(4^n/n^{1.5})$ ordered trees on n nodes.

Proof

Partition the $\binom{2(n-1)}{n-1}$ possible BP sequences in equivalence classes under rotation of the string. Take a random element of any class. Find the prefix with the largest value of $\texttt{count}_{)}(i) - \texttt{count}_{(}(i)$, and consider the rotation starting after this prefix. This is a BP. Thus, each class has size $\leq 2(n-1)$, and contains at least one BP:

$$\#\texttt{trees} \geq \frac{1}{2(n-1)}\binom{2(n-1)}{n-1} = \Omega\left(\frac{2^{2n}}{n^{1.5}}\right).$$

Space lower bound

Theorem 4.

We need at least

$$\frac{1}{n}\log_2(4^n/n^{1.5}) = 2 - o(1)$$

bits per node, and this is tight.

Problem 1 Goal for the lecture.

Represent trees using $2 + o(1)$ bits per node, while supporting common operations.

Aside: Catalan numbers

Theorem 5.

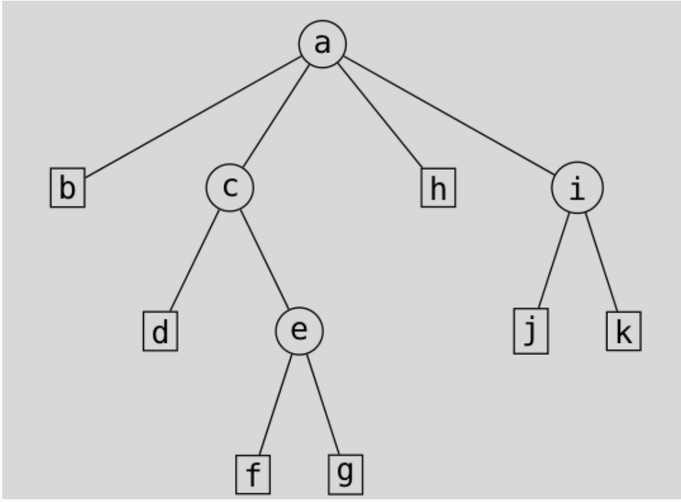
The *exact* number of ordered trees on n nodes is the *Catalan number*:

$$C_{n-1} := \frac{1}{n}\binom{2(n-1)}{n-1} \sim \frac{4^{n-1}}{n^{3/2}\sqrt{\pi}}.$$

Aside: Bijection to binary trees

Example 1 left-child right-sibling binary tree.

- Bijection between ordered trees and binary trees.
- Given a binary tree, transform it to an ordered tree like this:
- a *left* child of a node is a *child* in the ordered tree.
- a *right* child of a node is a *sibling* in the ordered tree.



Level Ordered Unary Degree Sequence (LOUDS)

Definition 2.

Start with `10`. Then append the degree of each node in level/BFS order in unary: 1 for each child, and then a 0.

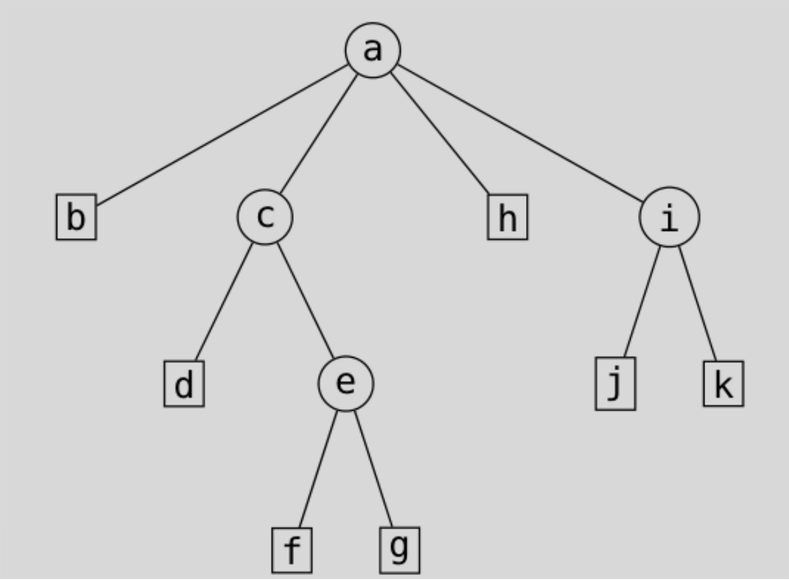
Example 2.

```
1011110011001100110000
a_bchiABdeCHjkIDfgEJKFG
```

- lowercase: *introduce* node as child.
- uppercase: end the list of children of the node.

$2n + 1$ bits:

- a 1 to add the node as child,
- an 'end' marker 0 for each node,
- an extra 0.



Operations

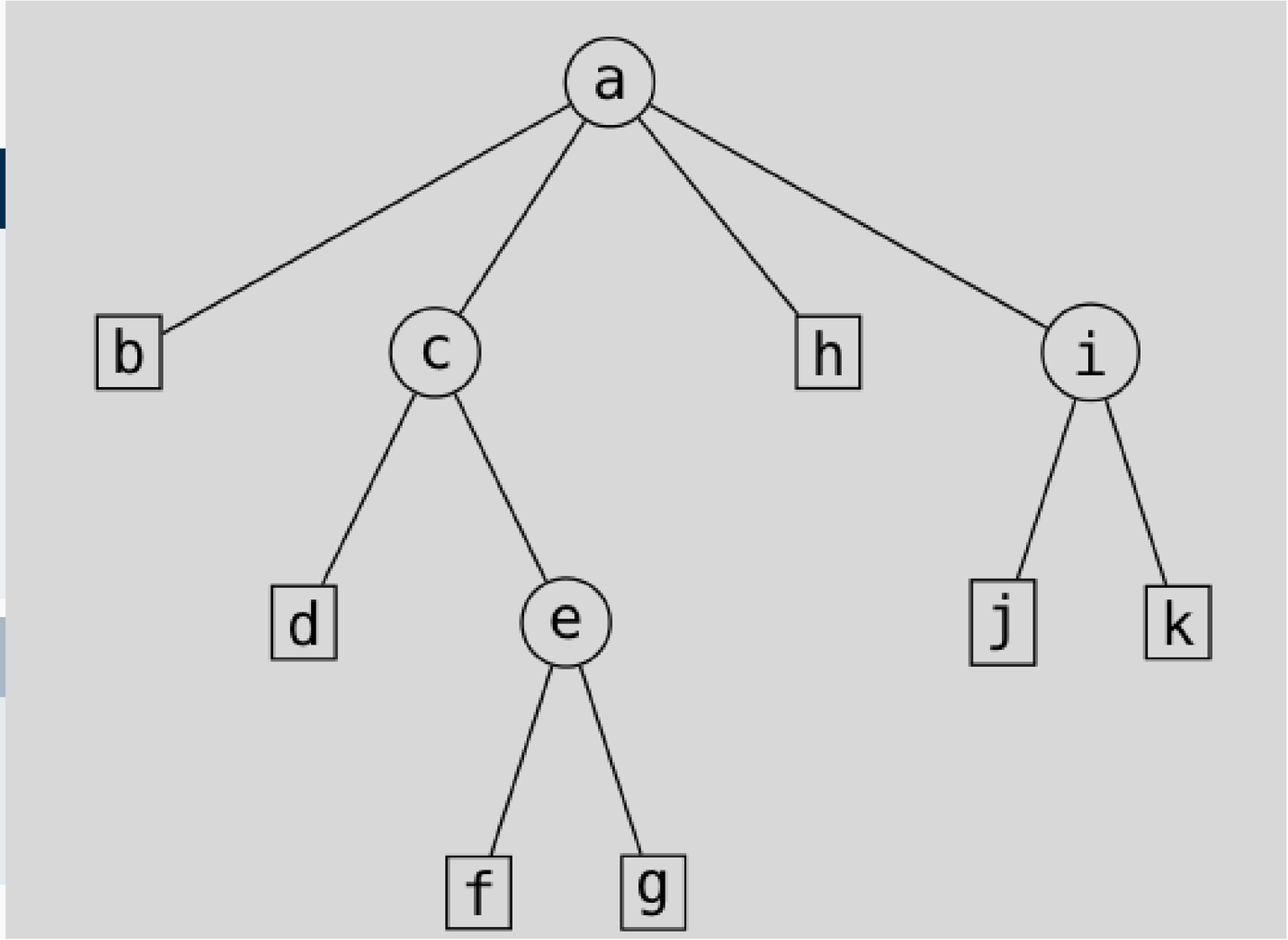
Question 2.

How to implement operations:

- degree,
- i'th child,
- parent,
- subtree size.

Example 3.

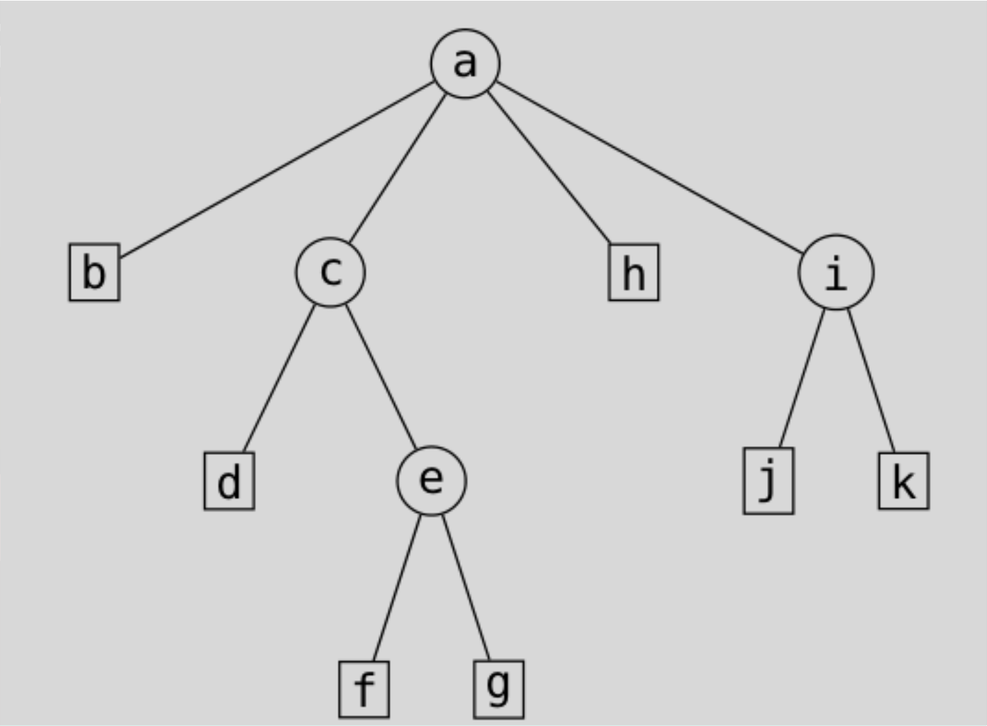
```
101111001100110011000 00
a_bchiABdeCHjkIDfgEJKFG
```



Implementing the operations

Example 4.

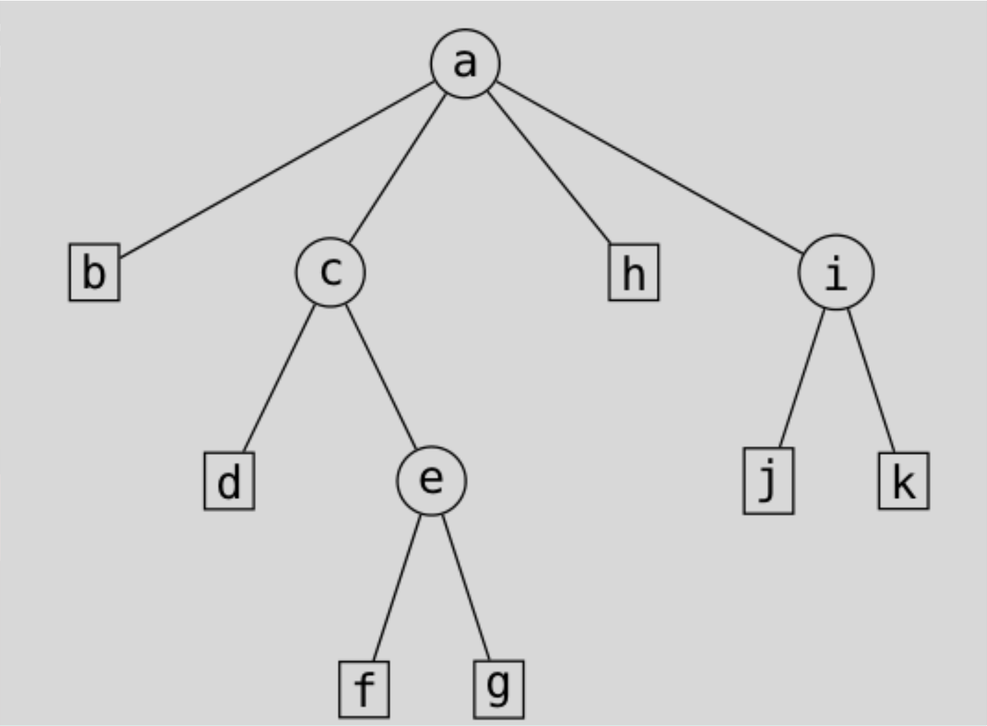
```
101111001100110011000000
a_bchiABdeCHjkIDfgEJKFG
```



Definition 3.

Number nodes 0 to $n-1$ in level/BFS-order.

Implementing the operations



Example 4.

```
101111001100110011000000
a_bchiABdeCHjkIDfgEJKFG
```

Definition 3.

Number nodes 0 to $n-1$ in level/BFS-order.

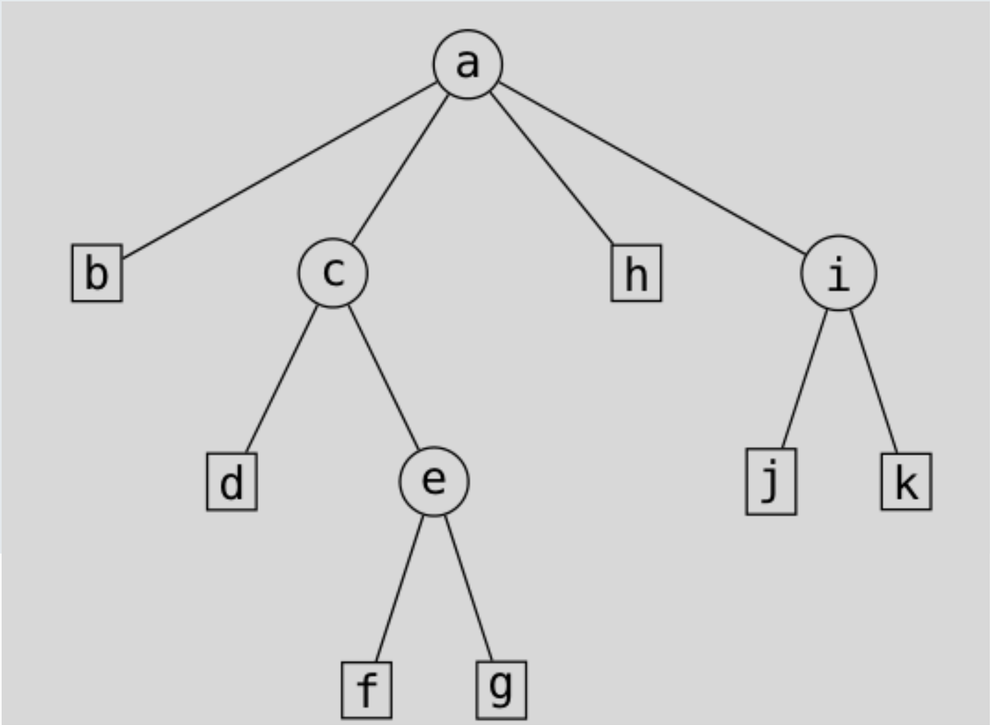
- Position of 0 of u: $p_0(u) := \mathsf{select}_0(u+1)$
- Position of 1 of u: $p_1(u) := \mathsf{select}_1(u)$
- Children of u: position $\mathsf{select}_0(u)+1$ to $\mathsf{select}_0(u+1)$
- Degree of u: $\mathsf{select}_0(u+1) - \mathsf{select}_0(u) - 1$.
- i'th child of u: (number of children before u) $+ i = \mathsf{rank}_1(\mathsf{select}_0(u)) + 1 + i$
- Parent of u: the node containing the u'th 1: $\mathsf{rank}_0(\mathsf{select}_1(u)) - 1$
  - Roughly inverse of child
- Subtree size: hard; data is all over the place

Balanced Parentheses (BP)

Definition 4.

Traverse tree in DFS-order. Write a (each time a node/subtree is first entered, and a) a node/subtree is left.

Example 5.



Balanced Parentheses (BP)

Definition 4.

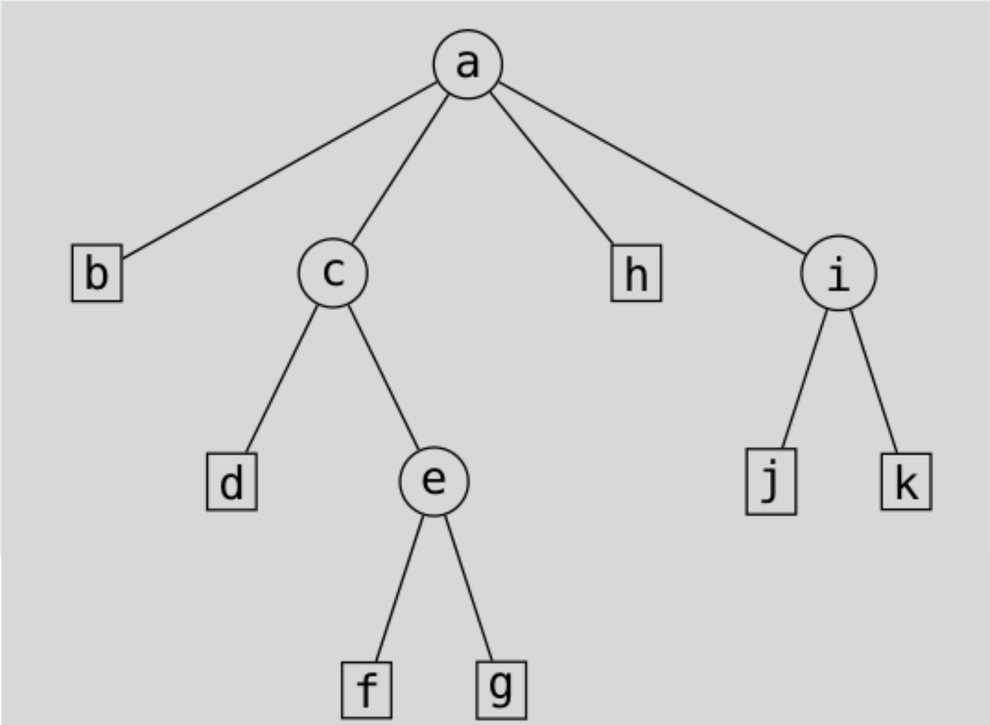
Traverse tree in DFS-order. Write a (each time a node/subtree is first entered, and a) a node/subtree is left.

Example 5.

`(()(()(()()))()(()()))`

`110110110100010110100 0` 10 instead of ()

`abBcdDefFgGEChHijJkKIA`



Balanced Parentheses (BP)

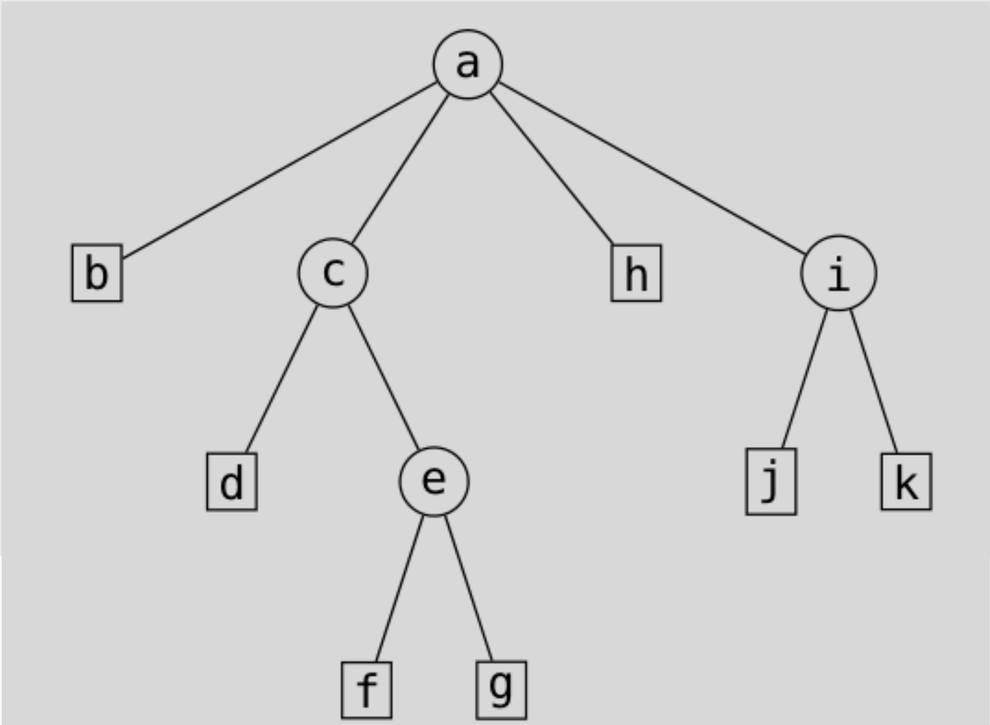
Definition 4.

Traverse tree in DFS-order. Write a `(` each time a node/subtree is first entered, and a `)` a node/subtree is left.

Example 5.

`(()(()(()()))()(()()))`
`110110110100010110100 0` 10 instead of ()
`abBcdDefFgGEChHijJkKIA`

- lowercase: *enter* subtree
- uppercase: *leave* subtree



Balanced Parentheses (BP)

Definition 4.

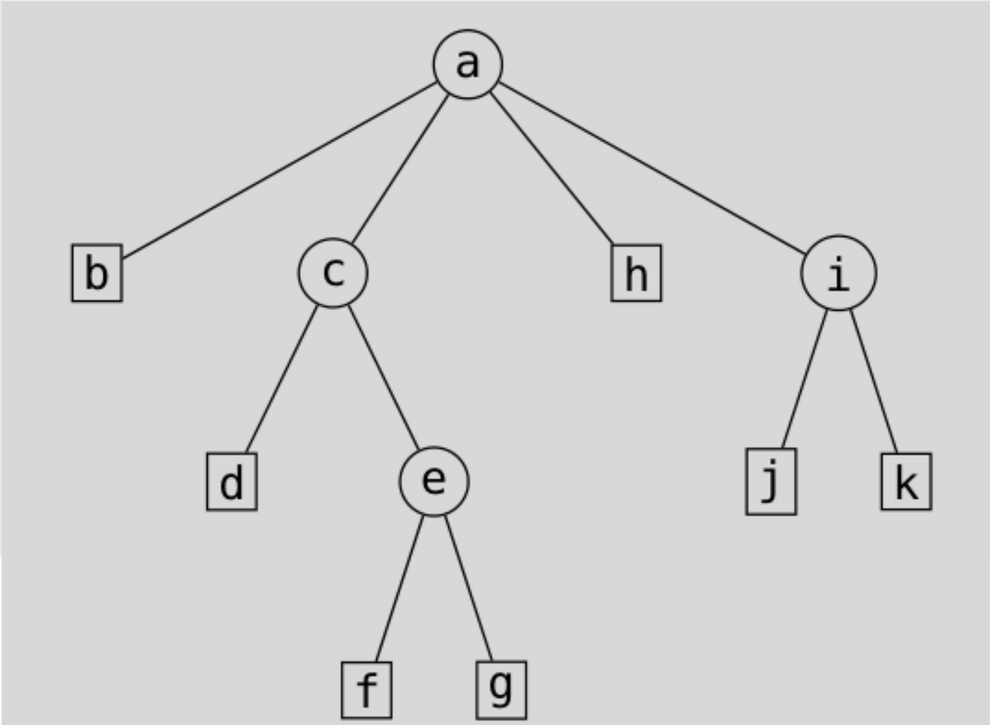
Traverse tree in DFS-order. Write a `(` each time a node/subtree is first entered, and a `)` a node/subtree is left.

Example 5.

`(()(()(()()))()(()()))`
`110110110100010110100`0 10 instead of ()
`abBcdDefFgGEChHijJkKIA`

- lowercase: *enter* subtree
- uppercase: *leave* subtree

$2n$ bits: a `(` and `)` for each node.
Note: the children of a are all over the place!
`rank` and `select` won't work here.



Additional operations on BP

Definition 5.

- $\mathsf{findclose}(\mathsf{i})$: given `(` at pos i, get the position of the matching `)`.
- $\mathsf{findopen}(\mathsf{i})$: given `)` at pos i, get the position of the matching `(`.
- $\mathsf{enclose}(\mathsf{i})$: given `(` at pos i, get the position of the closest `(` left of it enclosing it.

Can all be done in $O(1)$ time and $o(n)$ space

Implementation: See Navarro and Sadakane (2014)

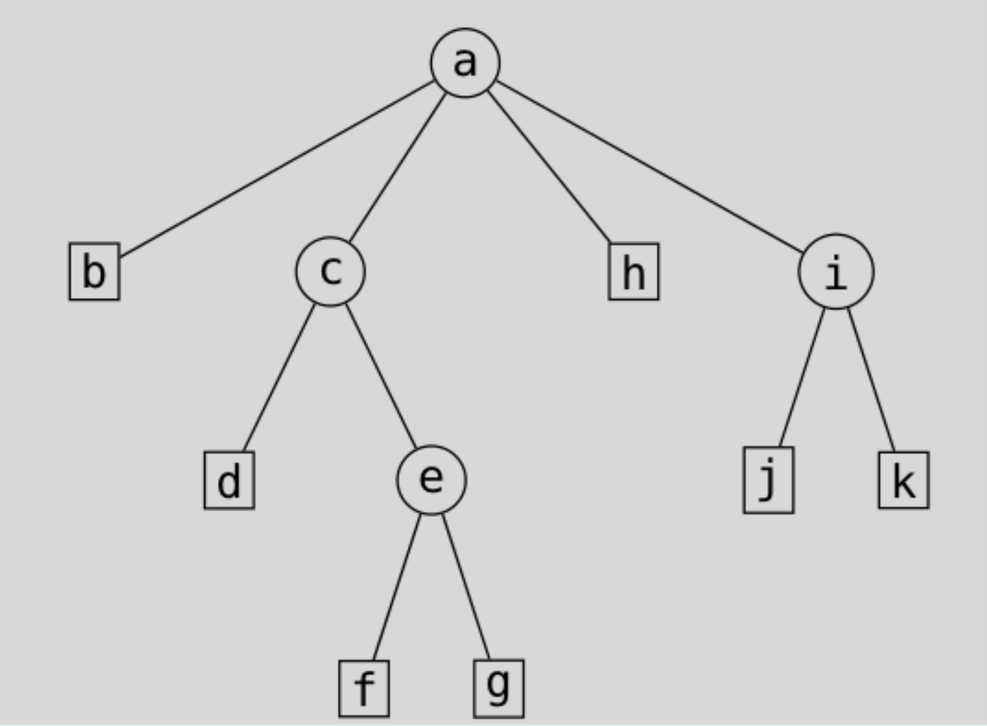
Implementing findclose and enclose

- Uses a segment tree (next week) over $\mathsf{excess}(i) = \mathsf{rank}_{(}(i) - \mathsf{rank}_{)}(i)$.
  - matches depth in the tree
- $\mathsf{findclose}(i) = \min\{j > i : \mathsf{excess}(j) = \mathsf{excess}(i-1)\}$
- $\mathsf{findopen}(i) = \max\{j < i : \mathsf{excess}(i) = \mathsf{excess}(j-1)\}$
- $\mathsf{enclose}(i) = \max\{j < i : \mathsf{excess}(i) = \mathsf{excess}(j-1) + 2\}$
- Each node of the segment tree stores the difference of (and) over its range, and the minimum depth node it contains.
- Further details omitted.

Implementing the BP operations

Example 6.

`(()(()(()()))()(()()))`
`1101101101000101101000` 10 instead of ()
`abBcdDefFgGEChHijJkKIA`



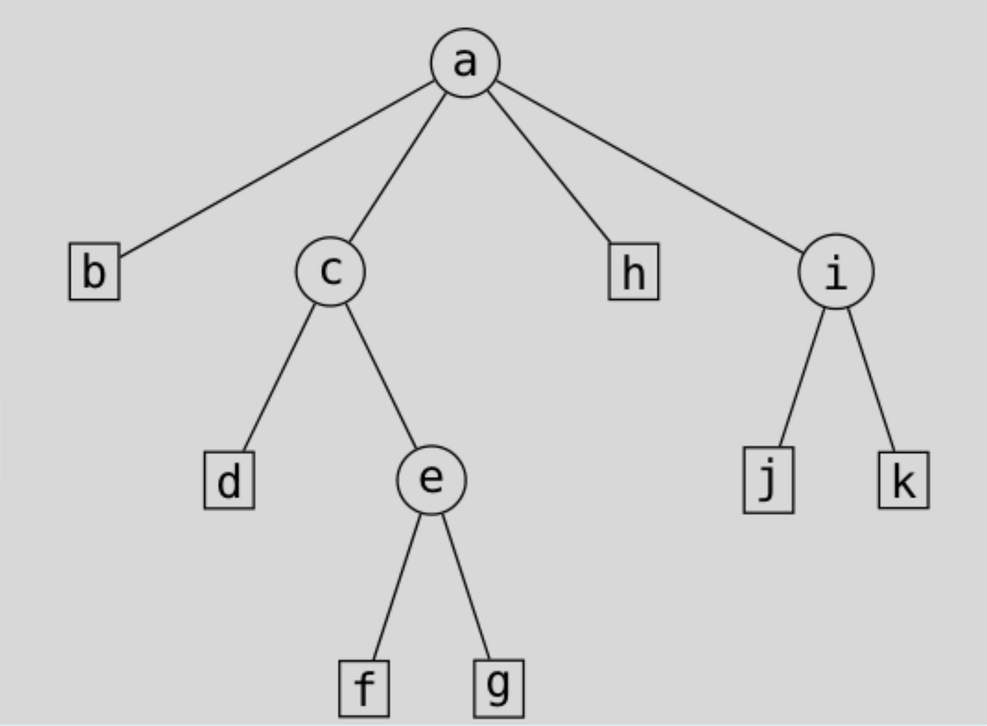
Definition 6.

Number nodes 0 to $n-1$ in DFS-order.

Implementing the BP operations

Example 6.

`(()(()(()()))()(()()))`
`110110110100010110100010` 10 instead of ()
`abBcdDefFgGEChHijJkKIA`



Definition 6.

Number nodes 0 to $n-1$ in DFS-order.

- Position of (of u: $p = \mathsf{select}_{(}(u)$
- Subtree size of u: $(\mathsf{findclose}(p) - p + 1)/2$
- Parent of u: $\mathsf{enclose}(p)$
- Degree, i'th child: hard (Navarro and Sadakane 2014)

Depth-First Unary Degree Sequence (DFUDS)

Definition 7.

Start with `1`. Then traverse the nodes in DFS order, and append for each its degree in unary: a `1` for each child and then a `0`.

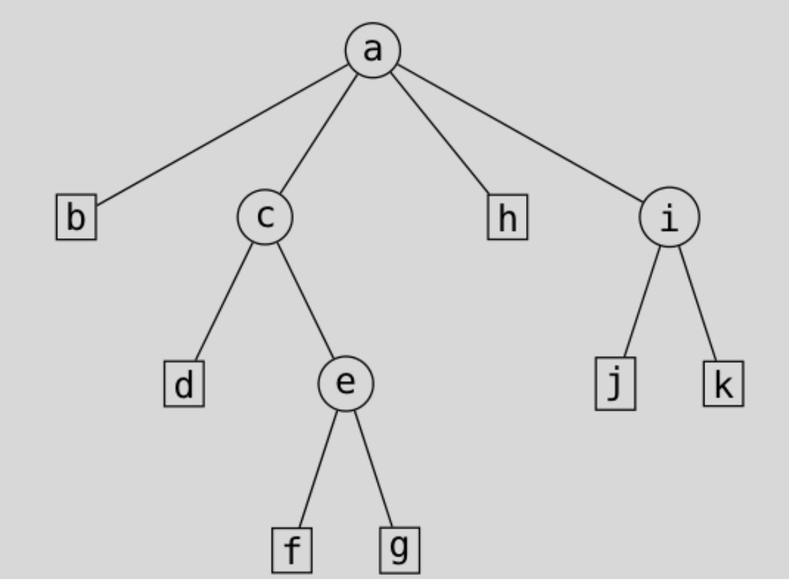
Then convert `1` to `(` and `0` to `)` to get a BP sequence.

Example 7.

```
((((()))(()))(()))))(()))
111110011001100001100 0
aihcbABedCDgfEFGHkjIJK
```

- lowercase: *introduce* node as child.
- uppercase: end the list of children of the node.

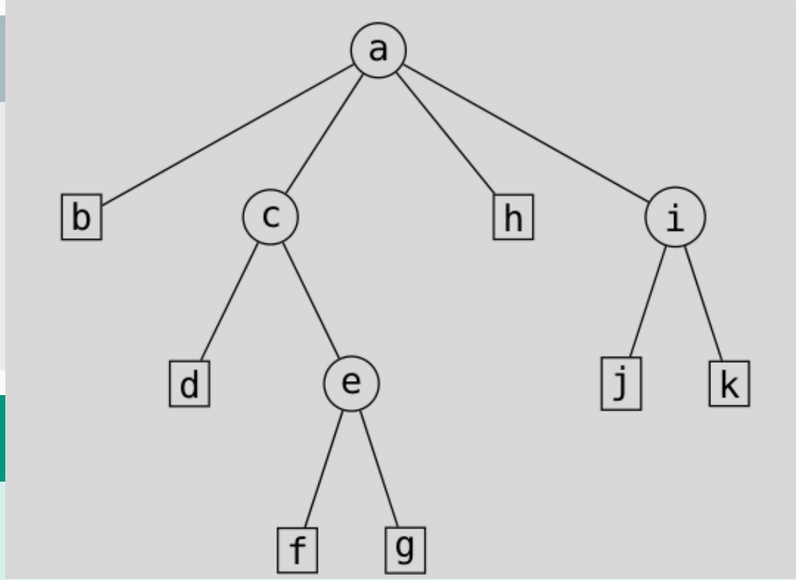
$2n$ bits: 1 to introduce each node,
0 to end the unary degree encoding of each node.



Implementing the DFUDS operations

Example 8.

```
((((()))(()))(()))))(()))
111110011001100001100 0
aihcbABedCDgfEFGHkjIJK
```

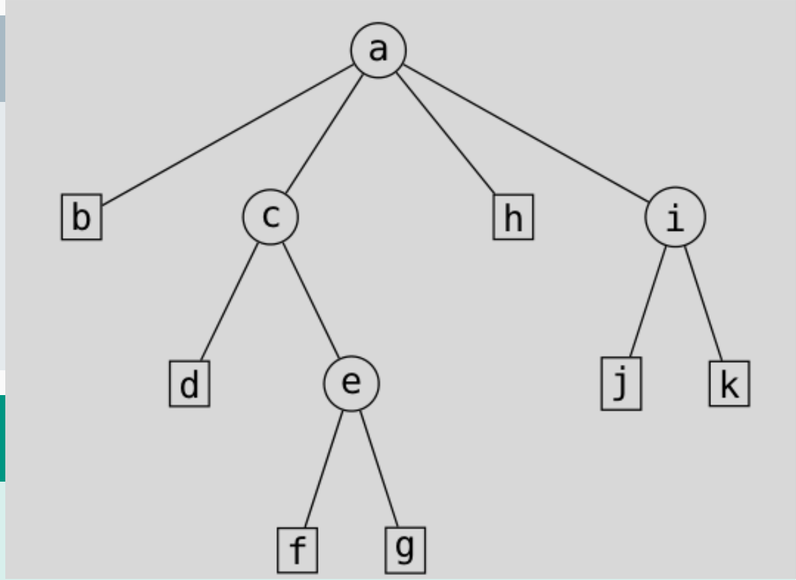


Definition 8.

Implementing the DFUDS operations

Example 8.

```
((((()))(()))(()))))(()))
111110011001100011000
aihcbABedCDgfEFGHkjIJK
```

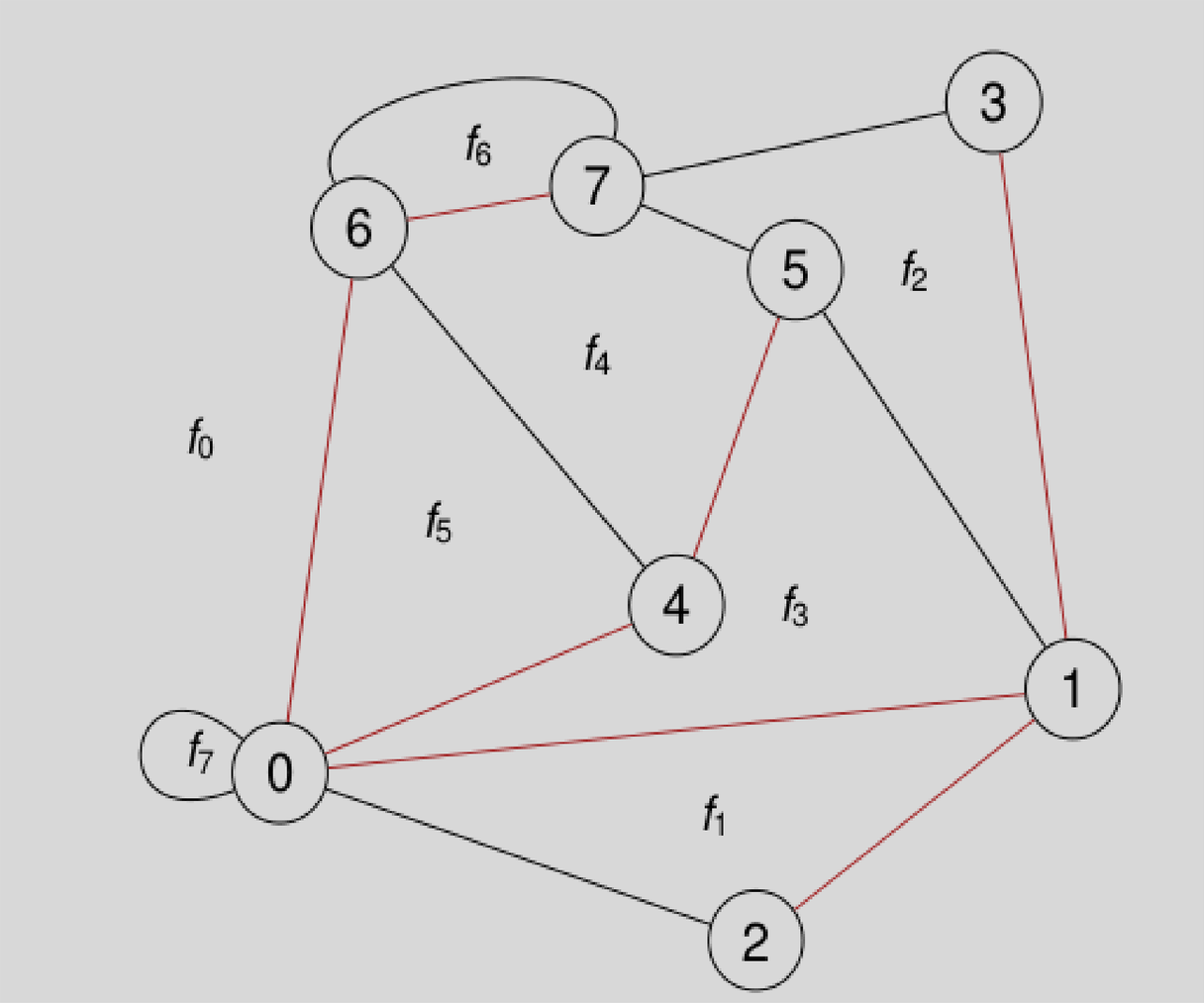


Definition 8.

- Position of 0 or) of u: $q = \mathsf{select}_0(u)$
- Degree of u: $\mathsf{select}_0(u) - \mathsf{select}_0(u-1) - 1$. (Edge case for $u = 0$)
- i'th child (position of first 1-bit): next after the matching): $\mathsf{findclose}(q - 1 - i) + 1$
- Parent of u: get preceding), then matching (, then succeeding): $\mathsf{select}_0(\mathsf{rank}_0(\mathsf{findopen}(\mathsf{select}_0(u-1))))$
- Subtree size of u, starting at first bit p: find succeeding) that leaves the subtree: $(\mathsf{findclose}(\mathsf{enclose}(p)) - p)/2 + 1$

Succinct planar graphs

- Euler's formula: $v - e + f = 2$,
  i.e. $e = (v-1) + (f-1)$
- Two spanning trees
  - one of nodes
  - one of faces
- DFS-Traverse the spanning tree T (red)
  - Start on outer face.
  - In each node go clockwise.
- Data:
  - For each edge we see, whether is in T
  - For each T-edge, whether we saw it before
  - For each non-T edge, whether we saw it before
- (Turán 1984; Ferres et al. 2020)



Further reading / sources

- Florian Kurpicz' slides from 2024
- Wikipedia on Catalan numbers
- Wikipedia on binary trees
- Wikipedia on binary tree equivalence
- Paper on succinct trees: Navarro, Gonzalo, and Kunihiko Sadakane. 2014. “Fully Functional Static and Dynamic Succinct Trees.” *Acm Transactions on Algorithms* 10 (3): 1–39. https://doi.org/10.1145/2601073.

Bibliography

Ferres, Leo, José Fuentes-Sepúlveda, Travis Gagie, Meng He, and Gonzalo Navarro. 2020. “Fast and Compact Planar Embeddings.” *Computational Geometry* 89 (August): 101630. https://doi.org/10.1016/j.comgeo.2020.101630.

Navarro, Gonzalo, and Kunihiko Sadakane. 2014. “Fully Functional Static and Dynamic Succinct Trees.” *Acm Transactions on Algorithms* 10 (3): 1–39. https://doi.org/10.1145/2601073.

Turán, György. 1984. “On the Succinct Representation of Graphs.” *Discrete Applied Mathematics* 8 (3): 289–94. https://doi.org/10.1016/0166-218x(84)90126-4.

Possible exam questions

- How many bit are needed to succinctly encode an ordered tree?
  - Why is this sufficient?
  - Why is this required?
- What are three different ways to succinctly encode an ordered tree?
  - How do they relate?
  - How do they differ?
  - Which operations are (not) easily supported by each variant?
- Why is it hard to support both 'subtree size' and 'i'th child' queries at the same time?
- In the LOUDS or DFUDS order, how does one get the degree of a node?